



TRABAJO PRACTICO 2

DISTRIBUIDOS

Analisis de los tiempos de comunicaciones
entre 2 computadoras distintas.

WRITE()

En la proxima diapositiva se muestra un grafico de los tiempos del write(), medido en NANOSEGUNDOS

ANALISIS DEL WRITE():

(10,100,1000 BYTES)

¿Que hace la sentencia write()? La sentencia write() se encarga de mover los datos desde el arreglo en memoria de usuario hacia el buffer de envio del socket, dentro del kernel. No espera a que los datos salgan ni a que lleguen.

¿Como esta implementado?

```
clock_gettime(CLOCK_MONOTONIC, &t0_write); // Inicia contador  
int bytes_enviados = write(sockfd, buffer, tam_buffer); // carga los datos en el buffer de envio  
clock_gettime(CLOCK_MONOTONIC, &t1_write); // Finaliza contador
```

Primeros 3 tamaños (10,100,1000): Hay algunas variaciones en los minimos, pero no son atribuibles al tamaño sino al ruido del muestreo. El promedio de tiempos son casi los mismos, esto se debe a la baja cantidad de bytes enviados, el mayor tiempo de procesamiento se lo lleva la syscall.

ANALISIS DEL WRITE():

(10.000, 100.000 BYTES)

10.000 bytes: Primer cambio en el promedio de los tiempos, crece casi el doble. El tiempo corresponde mayormente a la transferencia de memoria desde el Espacio de usuario de kernel (memcpy) y a la asignacion de las estructuras internas del kernel (sk_buff). Trabajando con esta cantidad de datos el trabajo es “deterministico”, es suficientemente grande para que el ruido del planificador no lo domine y suficientemente chico como para no chocar con algun limite del buffer.

100.000 bytes: Viendo el promedio se nota como sube ~4 veces mas que el anterior. Lo mas interesante de este tamaño es el minimo, el cual es muy parecido a trabajar con 10k (conclusion: en algunos casos trabajar con 10k es lo mismo que trabajar con 100k). Ademias la dispersion explota, del minimo 176 microsegundos a 1,12 milisegundos haciendo una distribucion muy asimetrica. Por que pasa esto? Lo mas probable es que en esos ~176 nanosegundos se produzca alguna interrupcion sumando esos cientos de microsegundos afectando al promedio. La mayoria de las mediciones estan cerca del coste real, y algunas se ven castigadas por las interrupciones.

ANALISIS DEL WRITE():

(1.000.000 BYTES)

1.000.000 bytes: En este caso el millon de bytes entra directamente en el buffer del socket, esto porque linux ajusta el tamaño dinamicamente. En caso de no ajustarlo hubiera tardado 200 veces mas ya que tendria que haber un while de por medio y esperando la capa TCP a que envíe los paquetes liberando el buffer para poder ingresar los nuevos datos.

READ()

En la proxima diapositiva se muestra un grafico de los tiempos del read(), medido en NANOSEGUNDOS

ANALISIS DEL READ():

¿Que hace la sentencia read()?: Genera una syscall() que mueve los datos del espacio de recepcion del socket (espacio kernel) a el buffer del programa (espacio usuario).

¿Como esta implementada?:

```
clock_gettime(CLOCK_MONOTONIC, &t0_read);  
int n_resp = read(sockfd, buffer_resp, 255);  
clock_gettime(CLOCK_MONOTONIC, &t1_read);
```

Es una linea de codigo pero atras hay un proceso grande, luego de hacer la sentencia write() se queda esperando en la sentencia read(), por lo tanto cuenta lo que tarda en transferir los datos del write() atravez de la capa de red con el protocolo TCP, el recibimiento de los mismos de parte del servidor con su propio read() y el envio de la respuesta del servidor hasta que se deposita en el espacio de recepcion del kernel y se realiza la syscall read() moviendo la respuesta del servidor del espacio de recepcion del socket al buffer del programa.

ANALISIS DEL READ(): (10,100,1000 BYTES)

Los primeros 3 tamaños (10,100,1000): tardan ~16 milisegundos, esto representa el costo fijo del enlace, dentro se encuentran las 2 transferencias de datos por la capa de transporte, la placa virtualizada de la VM y el procesamiento de las syscalls() de ambas computadoras. El tiempo de transmision no es lo suficiente como para afectar a los tiempos, todo lo que se mide aca es el RTT, el tiempo que tarda 1 paquete en ir y volver.

ANALISIS DEL READ(): (10,100,1000 BYTES)

10.000: Tomando a los ~16 milisegundos como el piso, con este tamaño se ve un +1,24 milisegundos por encima del piso, el volumen de datos deja una marca visible. Con este tamaño el envío no puede hacerse en 1 solo paquete TCP (~1460 bytes), se debe fragmentar en ~7 segmentos. Igualmente el piso sigue representando un 93% del total.

100.000: Con este tamaño ya el piso representa un 58% del total, el otro 42% representa a la transmisión que es proporcional al volumen de datos. Los datos 100.000 bytes tienen dividirse en ~69 segmentos de 1460 bytes y cada paquete tiene que: ganar el acceso al canal, ser transmitido y recibir la confirmación de nivel de enlace (ACK).

1.000.000: Con este tamaño se reduce significativamente el porcentaje que ocupa la latencia, pasa a un 9,6%, el resto del tiempo representa al tiempo de transmisión de los ~685 segmentos, con respecto al ancho de banda (bytes / tiempo de transmisión) es 6,67 MB/s, un 22% menos que en 100.000 bytes esto quiere decir que la tasa efectiva cae en transferencias largas ya sea tanto por la cantidad de segmentos como los eventos de interferencia y retransmisiones que se pueden generar.



